

Proporsal for Microprocessor Lab

Zihan Liu

Abstract—This proposal presents a human-adaptive, timing-based input decoder to be implemented on a PIC18F87K22 microcontroller in PIC18 assembly. The system estimates characteristic timing parameters (pulse duration and inter-symbol gap) from recent input events using lightweight exponential smoothing, then derives adaptive thresholds for binary classification. A Morse-code-style input is used solely as a demonstration workload. Performance will be evaluated against a fixed-threshold baseline using UART logs to assess convergence, accuracy, robustness, and responsiveness without requiring user-specific calibration.

I. INTRODUCTION AND MOTIVATION

THIS project proposes a human-adaptive, timing-based input decoder for embedded systems. Human-operated inputs exhibit substantial timing variability due to user behaviour, experience, and fatigue; fixed timing thresholds can therefore be brittle. The objective is to implement a decoder that estimates characteristic timing parameters from recent input history and adapts classification thresholds in real time. The implementation targets a PIC18F87K22 microcontroller and is written in PIC18 assembly, with minimal external electronics. A Morse-code-style input is used solely as a demonstration workload, not as a design constraint.

The technical objectives are:

- implement event-timed acquisition of pulse and gap durations on the PIC18F87K22,
- implement lightweight adaptive estimation suitable for assembly-level execution,
- evaluate the adaptive decoder against a fixed-threshold baseline.

The plan is to build a minimal, timing-driven decoder because timing variability is the primary failure mode of fixed-threshold inputs. Using assembly on the PIC18F87K22 forces careful control of timing, memory, and interrupt latency, which is central to the engineering challenge. A Morse-style workload is chosen because it provides a clear, measurable short/long timing structure without constraining the underlying design.

II. DESIGN

A. System Overview

The system measures input pulse durations and inter-symbol gaps, then classifies each pulse into one of two symbolic categories (short/long) using adaptive thresholds. Symbol sequences may be decoded into Morse characters for demonstration, but the decoding stage is modular and independent of the timing analysis logic. The architecture is designed to remain applicable to other timing-driven human interfaces.

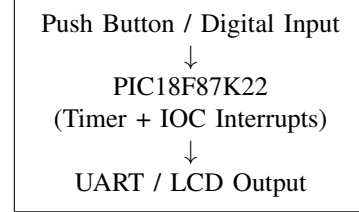


Fig. 1. Hardware-level signal flow.

B. Key Development Elements

To keep the plan focused, development is limited to three core elements:

- **Event timing capture:** reliable measurement of press and release events with debouncing and timestamping
- **Adaptive estimation and classification:** lightweight timing models and thresholding logic that run in assembly
- **Logging and evaluation:** UART output of event timings and decisions for offline analysis

C. System Operation Explanation

The system operates as a deterministic timing pipeline:

- 1) A timer and input-change interrupt capture edge times; a debounce guard rejects short intervals.
- 2) Each duration is computed, tagged as pulse or gap, and queued in a ring buffer.
- 3) The estimator updates the base unit and threshold; the classifier labels the pulse short or long.
- 4) The main loop logs durations, estimates, and labels over UART; optional Morse decoding remains separate.

This flow ensures the time-critical capture happens in interrupts, while estimation and logging remain lightweight and predictable.

D. Hardware Design

- **Microcontroller:** PIC18F87K22
- **Development board:** EasyPIC Pro
- **Input:** single push-button or digital logic input with software debouncing
- **Output:** UART serial output for logging and evaluation; LCD for real-time feedback

External electronics are kept minimal; timing and classification are handled in software using on-chip timers and interrupts. Timer selection is driven by the need for both resolution and range: a 16-bit timer with a suitable prescaler provides sub-millisecond granularity while allowing multi-second gaps when combined with an overflow counter.

1) Hardware Block Diagram:

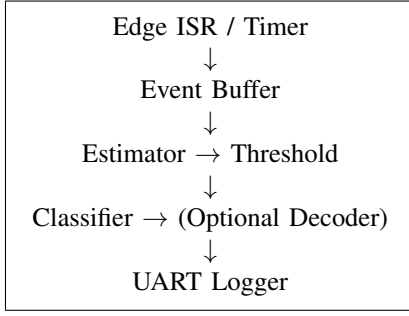


Fig. 2. Software module flow.

E. Software Design

The system is implemented in PIC assembly language using a top-down, modular design. A hardware timer provides a consistent time base, and interrupts are triggered on input state transitions. Interrupt Service Routines timestamp events, and pulse durations and inter-symbol gaps are computed and stored in RAM for real-time processing. The software is separated into acquisition, estimation, classification, optional decoding, and logging stages.

1) *Software Architecture Explanation*: The software is structured to isolate time-critical tasks from slower processing:

- **ISR layer**: timestamp edges, compute durations, enqueue events
- **Estimator layer**: update pulse and gap estimates (fixed-point)
- **Classifier layer**: compare pulse duration to adaptive threshold
- **Decoder (optional)**: map symbol sequences to Morse characters
- **Logger**: UART output of durations, estimates, and labels

This separation allows the ISR to remain minimal and deterministic while most processing occurs in the main loop.

2) *Software Block Diagram*:

3) *Adaptive Timing Estimation (Statistical Adaptation)*:

Running estimates of timing parameters are maintained using exponential smoothing:

$$T_{\text{est,new}} = \alpha T_{\text{est,old}} + (1 - \alpha) T_{\text{measured}}$$

Here T_{est} is a characteristic timing parameter (pulse or gap), T_{measured} is the latest measurement, and $\alpha \in (0, 1)$ trades responsiveness for stability. The method uses constant memory and simple arithmetic, making it suitable for PIC18 assembly. The value of α will be selected empirically. To avoid division in assembly, the update will be implemented in fixed-point (e.g., Q8.8) with α chosen as a power-of-two fraction so that multiplication reduces to shifts and adds. Separate estimates are maintained for pulse length and inter-symbol gap, allowing the system to adapt to both press duration and pause duration independently.

4) *Adaptive Threshold Generation*: Classification thresholds are derived from estimated timing parameters rather than fixed constants. The gap estimate defines the base unit time T_u , and the pulse threshold is set as a multiple of T_u (e.g., $2T_u$) so short and long presses remain separable across user speeds.

During start-up, T_u is seeded from the first few gaps; until enough events are collected, the classifier can defer decisions or use a conservative default.

5) *Symbol Classification and Optional Decoding*: Each input pulse is classified into one of two symbolic categories based on its duration relative to the adaptive threshold. This binary classification keeps the design simple while remaining expressive enough for demonstration. The optional Morse decoding stage remains separate from timing analysis to preserve modularity.

III. PROJECTED PERFORMANCE AND ASSESSMENT CRITERIA

The system will be assessed against a fixed-threshold baseline using the following criteria:

- **Convergence behaviour**: number of events required for estimates to stabilise
- **Classification accuracy**: correct classification rate across different user speeds
- **Robustness**: performance under irregular timing and transient disturbances
- **Responsiveness vs. stability**: adaptation speed relative to noise sensitivity

The adaptive decoder is expected to maintain higher accuracy when users change speed and to recover more quickly after timing disruptions, at the cost of a short convergence period after start-up or sudden changes. UART logs will be used to compute these metrics and compare adaptive versus fixed-threshold performance.

A. Performance Methodology Expansion

Evaluation will use controlled sessions where users enter the same short symbol sequence at slow, medium, and fast speeds. The UART log records raw durations, estimates, and classifications. Convergence is the number of events to remain within a band (e.g., $\pm 10\%$), accuracy compares output to the intended sequence, and robustness is measured by recovery after deliberate irregularities. A fixed-threshold decoder with identical logging serves as the baseline.

IV. SCOPE AND FEASIBILITY

To ensure feasibility within the project timeframe, the scope is intentionally constrained:

- **Included**: single-channel input, binary classification, adaptive thresholding with lightweight statistics, UART logging
- **Excluded**: machine learning algorithms, complex analogue conditioning, multi-channel or high-speed input processing

V. EXPECTED OUTCOMES

The project is expected to show improved robustness without explicit calibration and to clarify trade-offs between adaptability, simplicity, and responsiveness on the microcontroller. This proposal defines a feasible, assembly-level adaptive timing decoder on the PIC18F87K22 and will test whether adaptive timing improves robustness over fixed thresholds under realistic user variability.